2025年 プログラミングB 第2回レポート課題

担当教員： 天野 辰哉
日付： 令和 7 年 11 月 06 日

課題の目的

再帰を理解すること。

課題内容「電卓」

プログラム

本PDF付録の解説「再帰を用いた式評価の方法」を参考にして、以下の条件を満たすプログラムを作成せよ。

（条件1） 第1回課題と同様の電卓プログラム。ただし、加算乗算に加えて減算は必須とし、括弧の入れ子にも対応すること。入力式の数字は二桁以上であっても対応すること。ただし、入力式の数字を正数に限定してもよい。除算は考慮しなくても良い。

（条件2） 再帰を用いること。

レポート

「再帰」の長所・短所について、第1回・第2回課題で作成したプログラムを題材に考察し、A4 2~3ページ程度でまとめたレポートを作成せよ。

提出物

ソースコードとレポート

作成したソースプログラムおよびレポート（PDFファイルとする）をCLE で提出すること。

ソースコード

第1回課題同様に `.c` ファイルをCLEの「第2回課題プログラム提出フォーム」に提出すること．
ソースコードの**文字コードをUTF-8**にすること．後述のテストケースを通過することをまず自分のPCで確認してから提出すること．

CLE で正常に提出できたことを確認後，課題1と同様に動作確認を slack で依頼すること。

例）

```
/動作確認 課題2-09B12004-天野辰哉-Linux
```

提出ファイルの名前は「学籍番号下 4 桁.c」とすること．例えば，学籍番号が 09B16099 であれば，ファイル名は 6099.c となる．正常にコンパイルでき，正しく動作するプログラムのみを受理する．

レポート

`.pdf` ファイルをCLEの「第2回課題レポート提出フォーム」
提出ファイルの名前は「学籍番号下4桁.pdf」とすること．

提出フォーマットの不備・提出先の誤りに十分注意すること．

動作確認 依頼方法

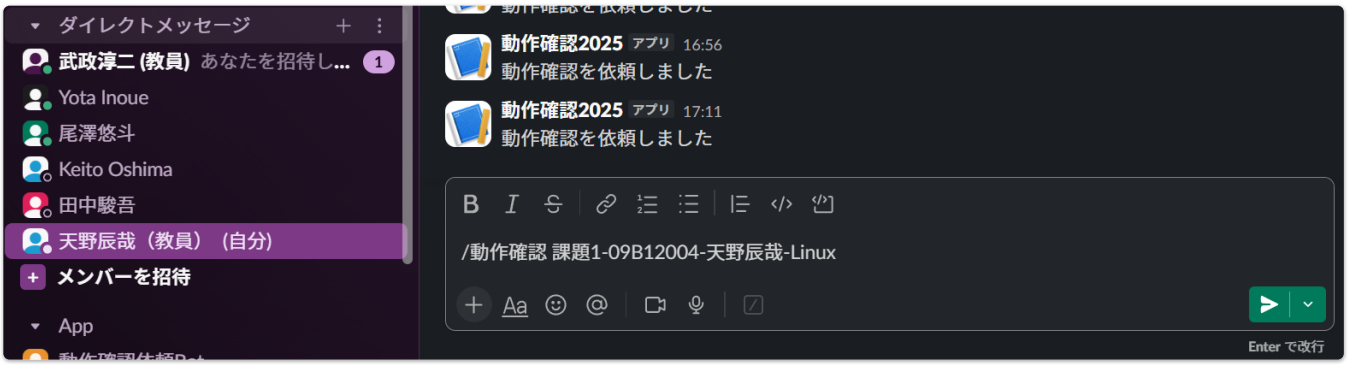
第1回課題と同様．まず上記の通りCLEの「第2回課題プログラム提出フォーム」ににソースコードのファイル（.c）を提出してください。

その後Slack上で「**自分あてのDM**」などで下記形式のコマンドを入力し動作確認Botへ動作確認を依頼。

```
/動作確認 課題X-学籍番号-氏名-Linux
```

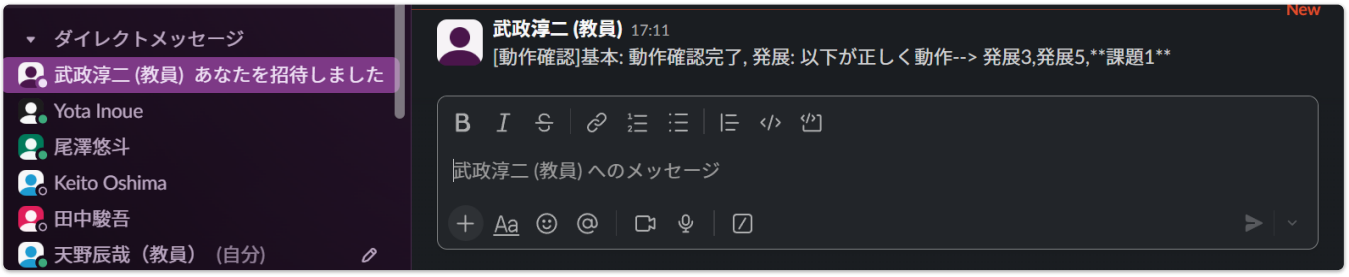
例）

```
/動作確認 課題2-09B12004-天野辰哉-Linux
```



任意のチャネルやDMでBotを呼び出し可能ですが、特に理由がない限り自分あてのDMで依頼ください。

動作確認結果はSlack上で「武政先生からのDM」として送られてきます。



発展機能に関するテストケースの結果も含まれます。最低限の提出要件としては**基本: 動作確認完了**であれば良い。

締切

本課題の提出期限は下記の通り。

提出期限：12 月 3 日（水）23:59

期限を過ぎての課題提出は大幅な減点を覚悟すること。

備考

質問は，Slackで受け付ける．教員あるいは TA から返事をする．ただし，すぐに返事があるとは限らない．課題提出について，「教員あるいはTA からの返事がなかったから締切に間に合わなかった」などの理由は認めない．他人のプログラムのコピーは厳禁.

他の学生に質問内容が見られても良いなら，slack の「#質問」チャンネルで質問をしても良い．むしろ，他の学生の参考になるので，slack での質問を強く推奨する.

Slack招待リンク：
https://join.slack.com/t/b2025co/shared_invite/zt-3e8mweeqp-dl17kxFpY7e689tLrY0EUQ

プログラムの仕様

以下の仕様を満たさないプログラムは、計算結果が正しい場合であっても受理しない場合がある。

入力

提出するプログラムは、入力式を標準入力で受け付けること。例えば、scanf関数やfgets関数などを用いると、標準入力で受け付けできる。不正な入力に対してはプログラムとして適切であると考えられる方法で処理すること。

本仕様を満たさない場合、計算結果が正しい場合であっても受理しない。例えば、`./0001.out 3+5*2`のように、実行時の引数として与える場合は、受理しない。

出力

提出するプログラムは、入力式の計算結果を数値の形式で出力すること。本仕様を満たさない場合、計算結果が正しい場合であっても受理しない。例えば、テストケースの基本aの正しい出力結果は、"82" である。"82です" など、不要な文字列 (この場合、"です") を含む場合は受理しない。

また、標準入力で式を受け付ける際にも不要な文字列を出力しないこと。例えば、式の入力前に、"Input formula: "などを出力するプログラムは受理しない。

入力式の末尾判定

受け取った入力式の末尾は、終端文字で判定すること。例えば、改行文字で判定するプログラムは受理しないことがある。

出席免除について（第1回とは異なります）

第2回課題について動作確認およびレポート提出が完了した学生で，下記の手続きを経て個別に承認を受けた学生はそれ以降の「演習（課題２）」回授業への出席を免除する．

- 動作確認およびレポート提出後、以降の第2回課題分演習授業への出席免除を希望する学生は**必ずSlackで天野宛のDMで出席免除希望旨を連絡すること**.
  - 教員が確認し返信があり初めて出席免除となる．動作確認のみを依頼し，無断で欠席したものは出席扱いにはならない．また教員からの返信なしで欠席をした場合も同様.

※例えば演習授業開始直前に動作確認とレポート提出を完了したとしても，教員がそれを確認し個別に返信できる十分な猶予がなければ，その回の出席が免状されることはない．時間に余裕をもって連絡すること.

発展課題

以下の発展課題は義務ではないが、取り組んだ場合はプラス評価する。レポートでこの取り組みや工夫について主張すること。

1. 入力として正整数だけでなく実数に対応すること。
2. 除算に対応すること。ゼロ除算（除数が0の除算）が発生した際にはゼロ除算エラーを英語で出力し、プログラムを終了させること。
3. 1桁の正整数を冪指数とする累乗に対応すること（100^3や28^7など）。ここでいう冪指数とは累乗n^cにおけるcである。累乗の演算子は'^'とする。累乗の計算は、C言語のライブラリを利用せず、自分でコーディングすること。
4. 文法チェックを行い、文法に沿わない入力が与えられたときにはその旨のエラーを出力し、プログラムを終了させること。その際、文法エラーの位置も出力すること。出力する文法エラーは、初めに検出したエラーのみでよい。今回の場合、"初めに"とは式を先頭から順に評価して初めに発見したエラーのことを指す。

テストケース

提出するプログラムは、提出前に基本a,bのテストケースをクリアすることを確認しておくこと。発展課題に取り組んだ場合は対応するテストケースのクリアを確認すること。

項番	入力式	結果
基本a	`5+(5*2-3)*(3+8)`	82
基本b	`(5+9*(1+3*2*(3+7)+4))*4`	2360
発展1	`5.3+1.1*4`	9.7
発展2a	`100/25+6`	10 （10.000なども可）
発展2b	`20/(10-(4+6))`	Zero division occurs.
発展3	`5^3+2^2`	129

項番	入力式	結果
発展4	`3+8*4-+(10-9)`	Syntax error occurs at the 7-th character.

付録：再帰を用いた式評価の方法

式の定義

課題1の式の文法は以下のように定義できる。

```
式 ::= 項 ['+' 項]
項 ::= 因子 ['*' 因子]
因子 ::= 定数 | '(' 式 ')'
定数 ::='0'| '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9'
```

ここで、「A B」は「Aの後ろにBが続く」ことを表す。また、「A ::= B」は「AはBである」と解釈する。「A | B」は「AまたはB」を表す。「[A]」は「Aの0回以上の繰り返し」と読む。「'x'」は「入力文字 'x'」と読む。

これらの定義によれば、上記の各式は次のように解釈できる。

- 式は1つ以上の項を足しあわせたものである
- 項は1つ以上の因子をかけあわせたものである
- 因子は定数であるか、または式を括弧で閉じたものである
- 定数は文字0から9のいずれかである

具体的な式を挙げて、上記の定義を当てはめると次のように解釈できる。

（例1）式 `2*3*4+5+6*7` の場合

- `2*3*4`、`5`、`6*7` がそれぞれ項にあたる。
- `2`、`3`、`4`、`5`、`6`、`7` はそれぞれの項の因子であり、それぞれ定数でもある。

（例2）式 `2*(3+4)+5` の場合

- `2*(3+4)`、`5` がそれぞれ項にあたる。
- `2`、`(3+4)`、`5` がそれぞれの項の因子にあたる。
- `3+4` はそれ自体が式である。（再帰構造）

「式」の定義の中に（間接的に）「式」が現れることに注意されたい。すなわち、上記の式は「再帰的に」定義されている。

再帰的な構造を持つデータを扱う場合、再帰処理を用いるとプログラムが自然でわかりやすくなる（再帰の利点）。みなさんが習った再帰的な構造を持つデータとしては、木構造が挙げられる。逆に、再帰的な構造を持つデータを再帰処理を用いずに処理しようとすると、アルゴリズムが複雑になり大変になる場合が多い。

再帰による解法

上記の再帰的な定義を基に、再帰処理によりこの課題を解く方法を説明する。ただし、本解説では減算・除算・二桁以上の数字は考慮しないことに留意すること。上記の式の定義を書き直すと、以下のようになる。

```
expression ::= term [ '+' term ]
term ::= factor [ '*' factor ]
factor ::= constant | '(' expression ')'
constant ::= '0' | '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9'
```

基本的な方針は以下に挙げる4点である。

- 入力された式を1文字ずつ参照しながら処理する。

- 常に「次の文字」が大域変数chに代入された状態を保ち（1文字先読みし）、chの値によって処理を分ける。
- expression、term、factor、constantのそれぞれに対して一つの関数を作る。関数はその評価結果を返す。
- A ::= B | C ならば、関数Aの中で関数BやCを呼び出す、というように各関数が連係して動く。

次ページはこの方針に基づいた式評価プログラムの例である。ただし、各関数の実装は省略してある。

```
#include<stdio.h>

char ch; /* 1文字先読み用 */

/** プロトタイプ宣言 ***/
int expression();
int term();
int factor();
int constant();

/*** 関数本体 ***/

int expression()
{・・・（略）・・・}

int term()
{・・・（略）・・・}

int factor()
{・・・（略）・・・}

int constant()
{・・・（略）・・・}

int main()
{
    printf("Input form: ");
    ch = fgetc(stdin);
    printf("%d\n", expression());
    return 0;
}
```